Football Intelligence Platform

What we built, in plain language - with the file names

What the project does. It answers two questions about football, using the record of every single touch in a season:

- **Who plays like this player?** Pick a player, get a ranked list of others who do similar things - plus the reasons why.
- **Which teams play alike?** Group the 20 teams by their style of play, and show what makes each group different.

It deliberately does **not** predict match results. It describes what happened, which is what a scout or analyst actually needs.

> **The data**
>
> StatsBomb Open Data for the Premier League 2015/16 season: **380 matches** and **1,313,773 individual events** (every pass, shot, tackle and pressure, each with pitch coordinates). It is free and legal to use with attribution. We chose depth over recency: newer free data has only summary totals, which cannot support the spatial features this project is built on.

Repository: github.com/SamiRadulov/football-intelligence-platform

1. The big picture: data flows downhill

Everything is built as a chain of layers. Each layer only reads from the one above it and never edits it. If a number looks wrong, you can walk back up the chain to find where it came from.

Layer	What lives there	Where
1. Raw	The original StatsBomb JSON files, completely unchanged	`data/raw/`
2. Staging	The same data flattened into clean tables (Parquet files)	`data/staging/`
3. Curated	Those tables loaded into a database you can query with SQL	`data/curated.duckdb`
4. Features	Football metrics calculated from the events (per 90 minutes, etc.)	`the mart_ tables`
5. App	The website that shows it all	`app.py + pages/`

> **Why the raw layer is never touched**
>
> Keeping an untouched copy of the source means every number in the app can be traced back to the file it came from. It also means a mistake in our code is always fixable by re-running, because we can never corrupt the source.

2. What we built, phase by phase

Phase 0 - Choosing the data

Before writing any real code we checked what data actually exists. A script counted the matches in all ~80 free competition-seasons, and we picked the only complete men's league season with full detail.

File	What it does
`scripts/data_audit.py`	Lists every available competition and counts its matches
`docs/data_audit.csv`	The result of that audit
`artifacts/feature_config.yml`	The project settings file: which season, which thresholds, which weights. Code reads settings from here instead of having them typed in.
`docs/METRIC_DICTIONARY.md`	The definition of every metric we calculate

Phase 1 - Downloading the data

A repeatable command that downloads all 380 matches (about 1.2 GB) and writes a manifest recording exactly what was fetched and when.

File	What it does
`scripts/download_data.py`	The command you run
`src/ingestion/statsbomb_open_data.py`	Knows the web addresses of the files, and downloads one file safely
`src/ingestion/raw_download.py`	Downloads everything and writes the manifest

Two details worth knowing: it skips files already downloaded, so re-running is safe; and it writes each file under a temporary name first, so an interrupted download can never leave a half-written file that looks valid.

Phase 2 - Turning JSON into clean tables

The raw files are deeply nested and awkward to work with. This phase flattens them into four tidy tables and, importantly, checks them.

File	What it does
`scripts/build_staging.py`	Runs the whole conversion
`src/transformations/matches.py`	One clean row per match
`src/transformations/players.py`	One row per player
`src/transformations/minutes.py`	Works out how many minutes each player actually played
`src/transformations/events.py`	Flattens all 1.3 million events
`src/validation/checks.py`	15 quality checks. The build fails if any of them break.

A bug the checks caught

One check compares each team's total minutes against 11 players x match length. It flagged 7 odd matches. Investigating showed our minutes formula was wrong: it counted time for players who went off injured and came back on. The fix was to add up each spell on the pitch separately, so gaps are excluded. A test now guards against it coming back.

Phase 3 - Turning events into football metrics

This is where raw touches become meaningful numbers: passes per 90 minutes, progressive carries, pressures, shot quality and so on. 33 metrics in total, each compared only against players in the same position group.

File	What it does
`scripts/build_player_features.py`	Runs it
`src/features/roles.py`	Sorts players into 5 groups: centre-backs, full-backs, central midfielders, wingers, strikers
`src/features/player_match.py`	Counts what each player did in each match
`src/features/player_season.py`	Turns those counts into per-90 rates and percentiles
`src/features/pitch.py`	The pitch zones (penalty box, Zone 14, halfspaces, wide channels)

Why position groups matter: 40 passes per 90 minutes is unremarkable for a centre-back and extraordinary for a striker. Every metric is therefore compared only within a group.

A bug found by sanity-checking

Every centre-back showed a 0% aerial duel win rate, which is obviously wrong. The cause: the aerial-won flag is buried inside each event type in the source data, not at the top level, so our code never found it. After the fix, Vincent Kompany and Per Mertesacker top the list, exactly as they should.

Phase 4 - The similarity engine

The heart of the recruitment tool. It compares the statistical shape of players and explains every answer it gives.

File	What it does
`src/similarity/feature_matrix.py`	Puts every metric on the same scale so they can be compared fairly
`src/similarity/engine.py`	Calculates similarity and explains each result
`scripts/find_similar.py`	Ask it a question from the command line
`scripts/validate_similarity.py`	Tests whether the answers are trustworthy
`METHODOLOGY.md`	The written explanation of how it all works

Two design rules. Candidates are filtered by position *before* scoring, so a centre-back can never be suggested for a winger search. And every result comes with the dimensions where the two players match

and where they differ.

Does it work? The football sanity check:

You search for	It returns
N. Kante	Idrissa Gueye (0.80), Jack Cork, Mousa Dembele - the other ball-winning midfielders
Mesut Ozil	Kevin De Bruyne (0.85), David Silva, Dimitri Payet - the other elite creators

And the trustworthiness tests:

Test	Result	What it means
Delete a random 20% of the season and re-run	81% of results stay the same	The answers reflect a whole season, not a few lucky matches
Raise the minimum-minutes bar	98% stay the same	The answers are not an accident of where we set the cut-off
Turn off the hand-tuned weights	92% stay the same	Honest finding: the weights matter less than we assumed

Phase 5 - Measuring team style

28 measurements of *how* a team plays: how much they keep the ball, how directly they attack, how high they press, how much they cross. Calculated for every team in every match.

File	What it does
`scripts/build_team_features.py`	Runs it
`src/features/team_match.py`	The 28 style measurements per team per match
`src/features/team_season.py`	Season averages, and how much each one varies from match to match

Why we keep the variation, not just the average: a team that plays the same way every week and a team that adapts to each opponent can have identical averages. The spread is what tells them apart.

> **A bug found by checking against reality**
>
> Set-piece shots came out at 54% of all shots, which is impossible - the real figure is about 25%. The cause was a definition problem: the source data labels a whole passage of play by how it started, so a shot 40 seconds after a throw-in still counted as a set piece. Now a set-piece shot must come within 10 seconds of a corner or free kick. Result: 25.5%, matching the published benchmark.

Phase 6 - Grouping the teams by style

The 28 measurements are boiled down to their main patterns, and teams are then grouped. Crucially, the groups were **named only after looking at what defined them**, never decided in advance.

File	What it does
`src/clustering/style_model.py`	The grouping maths

File	What it does
`scripts/build_team_clusters.py`	Run with --explore first to inspect the options, then again to save the model

The three styles it found, and who was in them:

Style	Teams	What defines them
Possession-dominant high press	Arsenal, Chelsea, Liverpool, Man City, Man Utd, Spurs	Keep the ball, play high up the pitch, press aggressively, few long balls
Direct and wide	Palace, Leicester, Norwich, Southampton, Sunderland, Watford, West Brom, West Ham	Move the ball forward quickly, many long passes and crosses, less possession
Deep build-up, low territory	Bournemouth, Villa, Everton, Newcastle, Stoke, Swansea	Play in their own half, pass backwards more, rarely reach the final third

The honest caveat we wrote down

The first group is exactly the big six. That means style and quality are tangled together, because better teams naturally have more possession. We state this openly in the methodology rather than waiting for someone to spot it. The counter-evidence: the direct and wide group contains both the champions (Leicester) and two relegated sides, so the groups are not simply the league table.

Phase 7 - The app

A six-page website that makes all of this usable.

File	The page
`app.py`	The entry point - start the app with this
`pages/home.py`	Overview, and what the tool does not claim
`pages/player_search.py`	Find similar players
`pages/player_comparison.py`	Compare two players side by side
`pages/team_style_map.py`	The map of team styles
`pages/team_profile.py`	One team in detail
`pages/methodology.py`	How it works, and the limitations
`src/app/data.py`	Loads data once and remembers it, so the app stays fast
`src/app/charts.py`	The charts

Charts show **percentiles** rather than statistical scores, because 84th percentile among centre-backs means something to a human and z = +1.02 does not.

Phase 8 - Polish (in progress)

Done so far: a small 2.4 MB database holding just the finished results, so the app can be put online without the 1 GB of raw data; and the whole codebase has been run through an automatic style checker (Ruff) and cleaned up.

File	What it does
`scripts/build_app_db.py`	Makes the small database for the live app
`data/sample/app.duckdb`	That database - the only data file stored in the repository

Still to do: automatic testing on every code change (GitHub Actions), screenshots, a model card, and putting the app online.

3. How we know it works

There are **78 automatic tests**. They run in under a minute and check the things that are easy to get quietly wrong.

Test file	What it protects
`tests/test_ingestion.py`	The download addresses are correct
`tests/test_minutes.py`	Minutes are counted correctly, including the injury-gap bug
`tests/test_events.py`	Events are flattened correctly, including the aerials bug
`tests/test_roles.py`	Players land in the right position group
`tests/test_player_match.py`	The football definitions: progressive pass, box entry, penalties excluded
`tests/test_similarity.py`	The similarity maths behaves sensibly
`tests/test_team_match.py`	Team measurements, including the set-piece fix
`tests/test_clustering.py`	Grouping finds groups that really are there
`tests/test_app_smoke.py`	Every page of the app loads without crashing

> **The pattern worth noticing**
>
> Three real bugs were found in this project: the minutes gap, the missing aerials, and the set-piece definition. None of them were found by staring at code. Each was caught by asking does this number look like football, and then investigating instead of assuming. That habit is the single most valuable thing here.

4. The commands

Run these from the project folder. Steps 1 to 5 only need running once.

```
cd "C:\Users\user\Documents\Claude\Projects\Football Intelligence platform"
```

Step	Command
1. Download the data (~1.2 GB, one-off)	`.venv\Scripts\python scripts\download_data.py`
2. Build the clean tables	`.venv\Scripts\python scripts\build_staging.py`
3. Build the player metrics	`.venv\Scripts\python scripts\build_player_features.py`
4. Build the team style metrics	`.venv\Scripts\python scripts\build_team_features.py`
5. Group the teams by style	`.venv\Scripts\python scripts\build_team_clusters.py`
Start the app	`.venv\Scripts\streamlit run app.py`

Step	Command
Ask a similarity question	`.venv\Scripts\python scripts\find_similar.py "Kante"`
Run all the tests	`.venv\Scripts\python -m pytest -q`

5. The written documents

File	What it contains
`README.md`	The front page: what it is and how to run it
`METHODOLOGY.md`	How the models work, the validation results, and the limitations
`docs/METRIC_DICTIONARY.md`	Every metric defined precisely, with its denominator
`DATA_SOURCES.md`	Where the data comes from and the attribution required
`artifacts/feature_config.yml`	All the analytical settings in one editable file

The one thing to remember. The value of this project is not that it uses clever maths. It is that every number can be traced back to a definition, every definition is written down, and every claim has been checked against what actually happened on the pitch. That is what makes it defensible when someone asks you about it.

Data provided by StatsBomb. Results are descriptive: similarity means two players do similar things, not that they are equally good.